DOCKER & NGINX

Hands-on Session Report

A step-by-step record of the session: activities, contributors, technical issues, resolutions, and outcomes.

Session overview

The team worked through three practical tasks: Docker container lifecycle management, running Nginx in Ubuntu and Docker, and hosting multiple applications through Nginx. The session was interactive: one participant shared the screen, the presenter explained the commands and concepts, and the group discussed and resolved issues as they appeared.

Completed tasks

TASK	FOCUS	SESSION RESULT
01	Docker container lifecycle	Created, inspected, stopped, restarted, and removed a container.
02	Nginx in Ubuntu and Docker	Verified Nginx, addressed AWS access rules, and launched an Nginx container.
03	Five application containers with Nginx	Created and tested five application containers and configured Nginx routing.

Note: This report is based on the supplied notes and meeting transcript. Where the transcript did not clearly preserve an exact command, port value, or participant attribution, the report describes the activity without guessing.

1. Session opening and how the meeting progressed

The meeting began with greetings and participant introductions. The team briefly revisited the activity shared by email and clarified that the goal was to complete the practical Docker tasks on their EC2/Ubuntu instances.

Screen sharing and facilitation

- **Srivalli** shared her screen and performed the practical demonstrations so the group could follow the commands.
- **Tejo Abhiram** guided the practical work, explained what each command did, and answered questions during the tasks.
- **Hemanth** acted as the scribe: capturing the session through transcription and preparing the written notes/documentation. Hemanth also helped with a container naming issue and raised questions during the Nginx configuration activity.
- **Harsha Anna** joined near the end to check progress and discuss the expected deliverables.

The session followed a practical pattern: the presenter explained the goal, Srivalli ran the commands while sharing her screen, participants followed along on their own instances, and the group paused to troubleshoot or clarify doubts.

2. Task 01 – Docker container lifecycle

Objective

Understand the basic lifecycle of a Docker container: create it, check its state, stop it, start it again, and remove it when it is no longer needed.

What the team did

- Checked whether Docker was installed and running on the Ubuntu/EC2 instance.
- Reviewed the Docker service status using `systemctl`.
- Created a container from an image such as Ubuntu or Nginx. The presenter first explained the basic `docker run` command, then demonstrated naming and detached/background execution.
- Used Docker listing commands to check running containers and all containers, including stopped ones.
- Stopped the container, started it again, and verified its state.
- Removed the container after stopping it, then checked that it no longer appeared in the container list.

Commands covered

```
sudo systemctl status docker
docker run [image]
docker ps
docker ps -a
docker stop [container]
docker start [container]
docker rm [container]
```

The session also introduced pause and unpause as container lifecycle operations. The exact command sequence was discussed as part of the task.

Issue and resolution

No major blocking issue was recorded for Task 01. The presenter corrected a spelling/command-entry issue during the initial image run and continued from the basic command so everyone could understand the process.

Outcome

Task 01 was completed. Participants practised creating, inspecting, stopping, restarting, and removing Docker containers.

3. Task 02 – Running Nginx in Ubuntu and inside Docker

Objective

Understand how an Nginx web server can run directly on Ubuntu and how the Nginx Docker image can be used to create and run an Nginx container. The team also checked how AWS security-group rules affect access to the web server.

Part A – Install and verify Nginx on Ubuntu

- Updated the Ubuntu package list and installed Nginx.
- Checked the Nginx service status to confirm that it was active.
- Used a local HTTP request with `curl` to verify that Nginx returned its default welcome page.
- Opened the EC2 public IP in a browser using HTTP and the web-server port to confirm external access.

Commands demonstrated

```
sudo apt update
sudo apt install nginx
sudo systemctl status nginx
curl localhost
```

Part B – Troubleshoot AWS access

Some participants could not reach the Nginx page from a browser. Tejo Abhiram guided the group to inspect the EC2 security group's inbound rules and confirm that web traffic was allowed on the Nginx port. The team checked the rule type/protocol and port configuration, then retried the public-IP address using HTTP rather than HTTPS. The connection worked after the rule/access details were corrected.

Important distinction: the Ubuntu service being active does not automatically make it reachable from the internet. The EC2 security group must also allow the relevant inbound traffic.

Part C – Run Nginx from a Docker image

- The team stopped the Nginx service that had been running directly on Ubuntu, so the Docker-based Nginx demonstration could be tested separately.
- Tejo Abhiram explained that Nginx is available as a Docker image and that a container can be created from that image.
- The group created/checked an Nginx container and discussed port mapping: a host port is mapped to the port inside the container so a browser can reach the service.
- The AWS inbound rule was reviewed again so the chosen host port could be accessed externally.
- The team used `docker ps` to inspect the running container and then tested the service through the browser.

Issue and resolution – existing container name

While creating another Nginx container, Srivalli encountered an error indicating that the requested container name was already in use. Hemanth suggested choosing a different, unused container name. After changing the name, the container could be created without that naming conflict.

Issue and resolution – application/server not reachable

Hemanth reported that an application page was showing a “site can't be reached” type of error. Following Tejo Abhiram's guidance, Hemanth restarted/reloaded the Nginx service. The application began working after the restart. The group discussed that Nginx needs to be reloaded or restarted after configuration changes so it can apply the updated configuration.

Outcome

Task 02 was completed. The team verified Nginx on Ubuntu, examined AWS inbound access, and practised running Nginx from a Docker container with port mapping.

4. Task 03 – Five application containers and Nginx reverse proxy

Objective

Create five application containers and use Nginx as a reverse proxy so users can reach different applications through URL paths, rather than exposing a separate public entry point for every application.

Step 1 – Review the existing applications

The team had already created two applications during an earlier session. The task was to continue the same pattern and create the remaining application containers, bringing the total to five.

Step 2 – Create the remaining containers

- The presenter guided the group to reuse the existing container-creation pattern while changing each application's name and port mapping.
- The team created the additional applications (App 3, App 4, and App 5) and checked the container list.
- The group discussed keeping each application's container name, port, and application response aligned. Each application should return its own identifying welcome message.

Step 3 – Resolve Hemanth's application mapping doubt

Hemanth noticed that requesting one application appeared to show another application's response. He asked how an application directory such as `app1/` related to the Nginx configuration and why the wrong application could appear. Tejo Abhiram reviewed the running containers and port assignments with him. The discussion identified that the application/container and port values needed to match; reusing the wrong port mapping could make one application's response appear where another was expected. Hemanth corrected the mapping/configuration and confirmed that it was working.

Step 4 – Configure Nginx routing

- The group started Nginx and worked with its configuration file in the terminal editor.
- The configuration was extended to include the application routes (for example, paths such as `/app1` and corresponding paths for the other apps).
- The presenter explained that Nginx receives the incoming request and forwards it to the appropriate application/container based on the configured route.
- The group ran an Nginx configuration test before applying the changes.
- After the configuration test succeeded, Nginx was reloaded so it could use the updated routing rules.

Commands demonstrated

```
docker ps
docker ps -a
curl localhost:[application-port]
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl status nginx
```

The transcript includes several port values in the 3000 range, but some spoken values were unclear in the recording. Use the exact port mapping from the team's task sheet/configuration rather than copying an uncertain value from this summary.

Step 5 – Test the applications

- The team tested application endpoints locally using `curl` and checked whether each one returned the expected application response.

- The Nginx configuration was tested, then reloaded.
- The team used the EC2 public IP and application paths in a browser to verify routing through Nginx.
- Srivalli created and tested five application/server entries. The group reported that the five applications were working during the practical verification.

What the group clarified about reverse proxy

A reverse proxy provides a single front door to multiple backend applications. Nginx listens for incoming web requests and routes each request to the correct application based on the URL path or configured rule. This avoids asking users to remember or directly access a different public port for every application. The backend applications can still run on their own internal/container ports.

Outcome

Task 03 was completed. Five application containers/servers were prepared and tested, and the group practised using Nginx configuration to route requests to the relevant application.

5. Issues faced and how they were resolved

ISSUE	WHO WAS INVOLVED	RESOLUTION / LEARNING
Nginx page could not be reached from the browser.	Tejo Abhiram guided the troubleshooting; the affected participant checked their EC2 setup.	Reviewed AWS security-group inbound rules, confirmed the web port/protocol, and retried using HTTP.
A container name was already in use.	Srivalli encountered it; Hemanth suggested the fix.	Used a different, unique container name before creating the new container.
An application page was not reachable after setup.	Hemanth reported it; Tejo Abhiram advised the next step.	Restarted/reloaded Nginx. The application became reachable afterward.
One application appeared to return another application's content.	Hemanth raised the doubt; Tejo Abhiram walked through container and port checks.	Checked the container list and aligned the app's name, port mapping, and Nginx route.
Configuration needed to be applied after editing.	The group, with Tejo Abhiram's guidance.	Ran the Nginx configuration test and reloaded Nginx to apply the updated configuration.

6. Roles and contributions

PARTICIPANT	CONTRIBUTION RECORDED IN THE SESSION
Tejo Abhiram	Led/explained the practical workflow, described the purpose of commands, guided AWS and Nginx troubleshooting, and clarified container, port-mapping, and reverse-proxy concepts.
Srivalli	Shared her screen, ran the demonstrations, created the Nginx/application containers, and tested the five application routes/servers.
Hemanth	Captured the session notes/transcript, helped Srivalli resolve the container-name conflict, raised questions about application/port mapping, and documented the work for the team.
Harsha Anna	Checked the team's progress near the end, discussed the expected evidence/screenshots, and clarified the team's reporting/deliverable expectations.
Other team members	Followed the practical tasks on their own instances, participated in discussions, checked their setups, and were asked to contribute screenshots and repository details.

7. Closing discussion and team deliverables

At the end of the practical work, the group reviewed the completed tasks and discussed how to submit evidence of the work.

- Participants were asked to capture screenshots showing their terminal commands and working application results.
- Hemanth shared an editable document structure so team members could add their names, commands, and screenshots in a consistent format.
- The group discussed creating or using a shared GitHub repository so team members could contribute their work. A Day One folder was mentioned for organizing the initial deliverables.
- Team members were asked to share their GitHub usernames/repository details so access could be provided.
- Harsha Anna was to be updated on the tasks completed, the team's progress, and the issues encountered.
- The group was encouraged to review the session concepts and complete any remaining documentation/screenshots after the meeting.

8. Key concepts to revise

- Docker image vs. container: an image is the packaged template; a container is a running or stopped instance created from an image.
- Container lifecycle: create/run, inspect, stop, start, pause/unpause, and remove.
- Port mapping: connects a host port to a port inside a container.
- AWS security groups: control which inbound network traffic can reach the EC2 instance.
- Nginx: a web server that can also act as a reverse proxy.
- Nginx reverse proxy: routes requests to the correct backend application based on configured paths or rules.
- Configuration validation and reload: test the Nginx configuration before reloading the service.

9. Final session summary

The team completed all three practical tasks: Docker container lifecycle management, running Nginx on Ubuntu and in Docker, and creating/testing five application containers behind Nginx routing. The session also provided practical troubleshooting experience with AWS inbound rules, duplicate container names, Nginx restarts/reloads, and application-to-port mapping.

The most valuable part of the session was the combination of hands-on execution and explanation. Participants did not only run commands; they discussed what each step achieved, checked results, identified configuration mistakes, and worked together to resolve them.

Prepared as a learning and activity report from the supplied session notes and transcript.